

Linked List FRQ Guide (C++)

Use this sheet when the FRQ involves a singly linked list. The goal is to identify the pattern fast, write the right pointer template, and collect full credit with fewer mistakes.

1. First 30 Seconds: What Are They Asking?

Before you write code, answer these questions on scratch paper:

1. What must I return?
 - Node* new head?
 - int size/count/value?
 - both, such as Node* plus int& size?
2. Can head change?
 - If the first node might be inserted or deleted, the answer is yes.
3. Am I only reading the list, or changing it?
 - Read only: traversal/count/search.
 - Change: insert/delete/reverse/split/merge.
4. What edge cases matter?
 - empty list
 - one node
 - deleting the head
 - deleting the last node
 - even number of nodes
5. Is memory management expected?
 - If you remove a node in C++, usually delete it.

If you do not know the exact function signature, write a reasonable one and state your assumption.

2. Basic Starter Code If They Do Not Provide It

If the FRQ does not give you a node class, write this:

```
struct Node {  
    int data;  
    Node* next;  
  
    Node(int d, Node* n = nullptr) : data(d), next(n) {}  
};
```

Generic function skeleton:

```

Node* solve(Node* head) {
    if (head == nullptr) {
        return nullptr;
    }

    // your logic here

    return head;
}

```

If the professor uses value instead of data, or ListNode instead of Node, just rename consistently.

3. Pattern Recognition Map

If the prompt says...	Main pattern	What you usually need
count, sum, search, contains, print	simple traversal	curr
delete, remove, erase	deletion with reconnect	curr, prev
insert at position, insert after, sorted insert	insertion with reconnect	curr, maybe prev, maybe newNode
middle	slow/fast pointers	slow, fast, maybe prev
kth from end	fast pointer gap	slow, fast
reverse	pointer flipping	prev, curr, next
split, partition	build 2 lists	extra heads/tails
merge two sorted lists	tail-building	dummy node or special head handling
keep track of size	count or decrement/increment	size or int& size

4. Full-Credit Checklist

Graders usually look for these:

- correct traversal logic
- correct pointer updates
- correct head-case handling
- proper return value
- no skipped nodes
- no lost part of the list
- delete when removing nodes
- correct update to size/count if required

A solution can be simple and still earn full credit if the pointer logic is safe and complete.

5. Universal Pointer Templates

A. Traverse the list

Use this when you are only reading values.

```
Node* curr = head;

while (curr != nullptr) {
    // use curr->data
    curr = curr->next;
}
```

B. Count nodes

```
int size = 0;
Node* curr = head;

while (curr != nullptr) {
    size++;
    curr = curr->next;
}
```

C. Search for a value

```
bool contains(Node* head, int target) {
    Node* curr = head;

    while (curr != nullptr) {
        if (curr->data == target) {
            return true;
        }
        curr = curr->next;
    }

    return false;
}
```

D. Delete the current node safely

This is the core deletion pattern.

```
if (curr == head) {
    head = head->next;
    delete curr;
    curr = head;
} else {
    prev->next = curr->next;
    delete curr;
}
```

```

    curr = prev->next;
}

```

Why it works:

- If deleting the head, move head first.
- If deleting a middle node, reconnect around it.
- After deletion, move curr carefully so you do not skip nodes.

E. Remove the first node with a matching value

```

Node* removeFirst(Node* head, int target) {
    Node* curr = head;
    Node* prev = nullptr;

    while (curr != nullptr) {
        if (curr->data == target) {
            if (curr == head) {
                head = head->next;
            } else {
                prev->next = curr->next;
            }

            delete curr;
            return head;
        }

        prev = curr;
        curr = curr->next;
    }

    return head;
}

```

F. Remove all nodes with a matching value

```

Node* removeAll(Node* head, int target) {
    Node* curr = head;
    Node* prev = nullptr;

    while (curr != nullptr) {
        if (curr->data == target) {
            if (curr == head) {
                head = head->next;
                delete curr;
                curr = head;
            } else {
                prev->next = curr->next;
                delete curr;
                curr = prev->next;
            }
        }

        prev = curr;
        curr = curr->next;
    }

    return head;
}

```

```

        }
    } else {
        prev = curr;
        curr = curr->next;
    }
}

return head;
}

```

Common reason this fails: updating prev after a deletion. Do not do that. prev should only move when you keep the node.

G. Insert at the front

```

Node* insertFront(Node* head, int value) {
    Node* newNode = new Node(value, head);
    return newNode;
}

```

H. Insert after a known node

```

void insertAfter(Node* curr, int value) {
    if (curr == nullptr) {
        return;
    }

    Node* newNode = new Node(value, curr->next);
    curr->next = newNode;
}

```

I. Insert into a sorted list

```

Node* insertSorted(Node* head, int value) {
    Node* newNode = new Node(value);

    if (head == nullptr || value <= head->data) {
        newNode->next = head;
        return newNode;
    }

    Node* curr = head;
    while (curr->next != nullptr && curr->next->data < value) {
        curr = curr->next;
    }

    newNode->next = curr->next;
    curr->next = newNode;
    return head;
}

```

J. Reverse a list

```
Node* reverse(Node* head) {
    Node* prev = nullptr;
    Node* curr = head;

    while (curr != nullptr) {
        Node* next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }

    return prev;
}
```

Remember this order:

1. save next
2. flip curr->next
3. move prev
4. move curr

K. Find the middle

Use slow/fast pointers.

```
Node* slow = head;
Node* fast = head;

while (fast != nullptr && fast->next != nullptr) {
    slow = slow->next;
    fast = fast->next->next;
}
```

After the loop, slow is at the middle. In an even-length list, this version lands on the second middle.

L. Remove the middle

```
Node* removeMiddle(Node* head) {
    if (head == nullptr || head->next == nullptr) {
        delete head;
        return nullptr;
    }

    Node* slow = head;
    Node* fast = head;
    Node* prev = nullptr;
```

```

while (fast != nullptr && fast->next != nullptr) {
    prev = slow;
    slow = slow->next;
    fast = fast->next->next;
}

prev->next = slow->next;
delete slow;
return head;
}

```

M. Find the kth node from the end

```

Node* kthFromEnd(Node* head, int k) {
    Node* slow = head;
    Node* fast = head;

    for (int i = 0; i < k; i++) {
        if (fast == nullptr) {
            return nullptr;
        }
        fast = fast->next;
    }

    while (fast != nullptr) {
        slow = slow->next;
        fast = fast->next;
    }

    return slow;
}

```

N. Merge two sorted lists

This is a common FRQ pattern if they want pointer manipulation without arrays.

```

Node* mergeSorted(Node* a, Node* b) {
    Node dummy(0);
    Node* tail = &dummy;

    while (a != nullptr && b != nullptr) {
        if (a->data <= b->data) {
            tail->next = a;
            a = a->next;
        } else {
            tail->next = b;
            b = b->next;
        }
        tail = tail->next;
    }
}

```

```

    if (a != nullptr) {
        tail->next = a;
    } else {
        tail->next = b;
    }

    return dummy.next;
}

```

6. How to Identify the Right Pattern Fast

If the task only asks for information

Examples:

- count the nodes
- return the largest value
- check whether x exists
- sum values at odd positions

Use a basic traversal:

```

Node* curr = head;
while (curr != nullptr) {
    // inspect curr->data
    curr = curr->next;
}

```

If the task removes nodes

Examples:

- remove all negatives
- delete the first duplicate
- remove the node after a target
- delete every even-valued node

Use curr and prev, and always think about the head case.

If the task inserts nodes

Examples:

- insert at the front
- insert after the third node
- insert in sorted order

You must reconnect pointers in the right order:

```
newNode->next = ...  
...->next = newNode;
```

If the task says middle, halfway, center, or median position

Use slow/fast pointers.

If the task says from the end

Use a fast pointer gap, then move both pointers.

If the task says reverse

Use the prev, curr, next flipping loop.

If the task says split or partition

You are probably building separate sublists and reconnecting them later.

7. End-to-End FRQ Workflow

Use this exact workflow during the test.

Step 1: Translate the prompt into a pattern

Ask:

- Is this traversal, delete, insert, reverse, middle, or merge?
- Does head change?
- Do I need prev?

Step 2: Write edge cases before the loop

Examples:

```
if (head == nullptr) {  
    return nullptr;  
}  
  
if (head == nullptr || head->next == nullptr) {  
    // special handling  
}
```

Step 3: Set up the right pointers

- traversal: curr
- delete: curr, prev
- middle: slow, fast, prev

- reverse: prev, curr, next

Step 4: Write the loop first, then the inside logic

This keeps the structure clean.

Step 5: Check the 5 danger zones

1. Did I handle head == nullptr?
2. Did I handle deleting or inserting at the head?
3. Did I move pointers correctly after deletion?
4. Did I lose part of the list by changing next too early?
5. Did I return the correct thing?

8. End-to-End Worked Example

Example prompt

“Remove the middle node of a singly linked list and keep track of the final size of the list.”

Step A: Identify the pattern

This is a deletion problem with a middle-node requirement.

- middle -> use slow/fast pointers
- delete -> need prev
- final size -> decrement size after removing

Step B: State reasonable assumptions

If the prompt does not specify exact details, these are reasonable:

- the function receives Node* head
- the current list size is passed by reference as int& size
- if the list has even length, remove the second middle
- if the list is empty, return nullptr
- if the list has one node, remove it and return nullptr

Step C: Write the solution

```
Node* removeMiddle(Node* head, int& size) {
    if (head == nullptr) {
        size = 0;
        return nullptr;
    }

    if (head->next == nullptr) {
        delete head;
```

```

        size = 0;
        return nullptr;
    }

    Node* slow = head;
    Node* fast = head;
    Node* prev = nullptr;

    while (fast != nullptr && fast->next != nullptr) {
        prev = slow;
        slow = slow->next;
        fast = fast->next->next;
    }

    prev->next = slow->next;
    delete slow;
    size--;

    return head;
}

```

Step D: Dry run it

Input:

1 -> 2 -> 3 -> 4 -> 5, size = 5

Pointer movement:

- start: slow = 1, fast = 1, prev = nullptr
- loop 1: prev = 1, slow = 2, fast = 3
- loop 2: prev = 2, slow = 3, fast = 5
- stop because fast->next == nullptr

Now:

- slow points to the middle node 3
- prev points to 2

Delete:

- prev->next = slow->next makes 2 -> 4
- delete 3
- size--

Output:

1 -> 2 -> 4 -> 5, size = 4

Step E: Even-length note

For:

1 -> 2 -> 3 -> 4

This version removes 3, the second middle, because of where `slow` lands.

If the professor wants the first middle instead, the slow/fast setup or loop condition may need to change. If the prompt is unclear, state your assumption.

Step F: Why this earns full credit

- handles empty list
- handles one-node list
- uses the right pattern for middle
- reconnects pointers correctly
- deletes removed node
- updates size
- returns the updated head

9. Common Mistakes That Cost Points

Mistake 1: Forgetting the head case

Wrong idea:

```
prev->next = curr->next;
```

This crashes if `curr == head` and `prev == nullptr`.

Mistake 2: Using `curr` after deleting it

Wrong idea:

```
delete curr;  
curr = curr->next;
```

Once deleted, `curr` is no longer safe to access.

Mistake 3: Advancing `prev` after deleting

If you remove `curr`, do not move `prev` forward. It should stay where it is.

Mistake 4: Losing the rest of the list during reverse

Wrong idea:

```
curr->next = prev;  
curr = curr->next;
```

Once you flip `curr->next`, the original next node is lost unless you saved it first.

Mistake 5: Forgetting to return the new head

This matters when:

- inserting at the front
- deleting the first node
- reversing the list

10. A Safe “If I Panic” Template

If the FRQ is about removing nodes based on a condition, this is the safest general form:

```
Node* solve(Node* head) {
    Node* curr = head;
    Node* prev = nullptr;

    while (curr != nullptr) {
        if (/* delete condition */) {
            if (curr == head) {
                head = head->next;
                delete curr;
                curr = head;
            } else {
                prev->next = curr->next;
                delete curr;
                curr = prev->next;
            }
        } else {
            prev = curr;
            curr = curr->next;
        }
    }

    return head;
}
```

If the FRQ is about reversing, this is the safest general form:

```
Node* solve(Node* head) {
    Node* prev = nullptr;
    Node* curr = head;

    while (curr != nullptr) {
        Node* next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
}
```

```
    return prev;  
}
```

11. Last-Minute Exam Checklist

Before you move on, scan your code and ask:

1. Did I choose the right pattern?
2. Does the head case work?
3. Are my loop conditions safe?
4. If I deleted a node, did I reconnect correctly and delete it?
5. If I reversed pointers, did I save next first?
6. Did I return the correct value?
7. Did I update size or count if the prompt asked for it?

If those answers are yes, your FRQ is probably in strong shape.